

```
1 @page
2 @model ERP.Pages.IndexModel
3 @{
4     ViewData["Title"] = "ERP";
5 }
6
7 <style>
8     .portal-hero {
9         margin-bottom: 1.5rem;
10     }
11
12     .portal-title {
13         font-size: 2.2rem;
14         font-weight: 700;
15         margin-bottom: 0.2rem;
16     }
17
18     .portal-subtitle {
19         color: #6c757d;
20         font-size: 1rem;
21     }
22
23     .portal-card {
24         border: 1px solid #e9ecef;
25         border-radius: 14px;
26         background: #fff;
27         box-shadow: 0 0.125rem 0.25rem rgba(0,0,0,.06);
28         height: 100%;
29     }
30
31     .portal-card .card-body {
32         padding: 1.2rem;
33     }
34
35     .kpi-label {
36         color: #6c757d;
37         font-size: 0.95rem;
38         margin-bottom: 0.4rem;
39     }
40
41     .kpi-value {
42         font-size: 2rem;
43         font-weight: 700;
44         line-height: 1.1;
45     }
46
47     .kpi-note {
48         color: #6c757d;
49         font-size: 0.9rem;
50         margin-top: 0.35rem;
51     }
52
53     .section-title {
```

```
54         font-size: 1.05rem;
55         font-weight: 600;
56     }
57
58     .simple-badge {
59         display: inline-block;
60         padding: 0.35rem 0.6rem;
61         border-radius: 999px;
62         font-size: 0.8rem;
63         font-weight: 600;
64     }
65
66     .badge-finished {
67         background: #d1e7dd;
68         color: #0f5132;
69     }
70
71     .badge-pending {
72         background: #fff3cd;
73         color: #664d03;
74     }
75
76     .badge-active {
77         background: #cfe2ff;
78         color: #084298;
79     }
80
81     .badge-other {
82         background: #e9ecef;
83         color: #495057;
84     }
85
86     .phase-badge {
87         display: inline-block;
88         padding: 0.28rem 0.55rem;
89         border-radius: 999px;
90         font-size: 0.78rem;
91         font-weight: 600;
92     }
93
94     .phase-before {
95         background: #fff3cd;
96         color: #664d03;
97     }
98
99     .phase-after {
100         background: #d1e7dd;
101         color: #0f5132;
102     }
103
104     .quick-links .btn {
105         padding-top: 0.8rem;
106         padding-bottom: 0.8rem;
```

```
107         font-weight: 600;
108     }
109
110     .table thead th {
111         white-space: nowrap;
112     }
113
114     .table td {
115         vertical-align: middle;
116     }
117 </style>
118
119 <div class="portal-hero d-flex flex-column flex-lg-row align-items-lg-
    center justify-content-between gap-3">
120     <div>
121         <div class="portal-title">ERP</div>
122         <div class="portal-subtitle">
123             Forenklet ERP-system for ordre, produksjonsdata og analyse
124             i Festo Smart Factory
125         </div>
126     </div>
127
128     <div class="d-flex flex-wrap gap-2">
129         <a class="btn btn-primary" asp-page="/CreateOrder">Opprett
130         ordre</a>
131         <a class="btn btn-outline-primary" asp-page="/
132         Reports">Rapport</a>
133         <a class="btn btn-outline-dark" asp-page="/Analyse">Analyse</a>
134     </div>
135
136 @if (!string.IsNullOrEmpty(Model.ErrorMessage))
137 {
138     <div class="alert alert-danger">@Model.ErrorMessage</div>
139 }
140
141 <div class="row g-3 mb-4">
142     <div class="col-12 col-md-6 col-xl-3">
143         <div class="portal-card">
144             <div class="card-body">
145                 <div class="kpi-label">Fullførte ordre</div>
146                 <div class="kpi-value">@Model.Kpi.CompletedOrders</div>
147                 <div class="kpi-note">Ordre med status Finished</div>
148             </div>
149         </div>
150
151     <div class="col-12 col-md-6 col-xl-3">
152         <div class="portal-card">
153             <div class="card-body">
154                 <div class="kpi-label">Aktive / Pending ordre</div>
155                 <div class="kpi-value">@Model.Kpi.ActivePendingOrders</
156                 div>
```

```

155         <div class="kpi-note">Ordre som ikke er fullført ennå</div>
156     </div>
157 </div>
158 </div>
159
160 <div class="col-12 col-md-6 col-xl-3">
161     <div class="portal-card">
162         <div class="card-body">
163             <div class="kpi-label">Gjennomsnittlig syklustid</div>
164             <div class="kpi-value">@Model.FormatDouble
165                 (Model.Kpi.AvgCycleSeconds, 1) s</div>
166             <div class="kpi-note">Fra start til ferdig sortert
167                 ordre</div>
168         </div>
169     </div>
170 </div>
171
172 <div class="col-12 col-md-6 col-xl-3">
173     <div class="portal-card">
174         <div class="card-body">
175             <div class="kpi-label">Gjennomsnittlig feilvarer</div>
176             <div class="kpi-value">@Model.FormatDouble
177                 (Model.Kpi.AvgQuantityProductFault, 2)</div>
178             <div class="kpi-note">QuantityProductFault per
179                 fullført ordre</div>
180         </div>
181     </div>
182 </div>
183 </div>
184 </div>
185
186 <div class="row g-3">
187     <div class="col-12 col-xl-9">
188         <div class="portal-card">
189             <div class="card-body pb-0">
190                 <div class="d-flex justify-content-between align-items-
191                     center mb-3">
192                     <div>
193                         <div class="section-title">Siste ordre</div>
194                         <div class="text-muted small">Oversikt over de
195                             nyeste ordrene i systemet</div>
196                     </div>
197                     <a class="btn btn-sm btn-outline-secondary" asp-
198                         page="/Analyse">Se trender →</a>
199                 </div>
200             </div>
201         </div>
202     </div>
203
204     <div class="table-responsive">
205         <table class="table table-hover align-middle mb-0">
206             <thead>
207                 <tr>
208                     <th>OrderId</th>
209                     <th>Testfase</th>

```

```

200         <th>Status</th>
201         <th>Opprettet</th>
202         <th>Start</th>
203         <th>Ferdig</th>
204         <th class="text-end">Syklustid (s)</th>
205         <th class="text-end">Feilvarer</th>
206     </tr>
207 </thead>
208 <tbody>
209     @if (Model.LatestOrders.Count == 0)
210     {
211         <tr>
212             <td colspan="8" class="text-muted p-3">Ingen ordre registrert ennå.</td>
213         </tr>
214     }
215     else
216     {
217         foreach (var o in Model.LatestOrders)
218         {
219             <tr>
220                 <td class="fw-semibold">@o.OrderId</td>
221                 <td>
222                     @if (o.TestPhase == "Before")
223                     {
224                         <span class="phase-badge phase-before">Before</span>
225                     }
226                     else if (o.TestPhase == "After")
227                     {
228                         <span class="phase-badge phase-after">After</span>
229                     }
230                     else
231                     {
232                         <span class="text-muted">-</span>
233                     }
234                 </td>
235                 <td>
236                     @if (o.Status == "Finished")
237                     {
238                         <span class="simple-badge badge-finished">Finished</span>
239                     }
240                     else if (o.Status == "Pending")
241                     {
242                         <span class="simple-badge badge-pending">Pending</span>

```

```

span>
243     }
244     else if (o.Status == "Active" ↗
245         || o.Status == "Running")
246     {
247         <span class="simple-badge ↗
248         badge-active">@o.Status</ ↗
249         span>
250     }
251     else
252     {
253         <span class="simple-badge ↗
254         badge-other">@o.Status</ ↗
255         span>
256     }
257 </td>
258 <td>@o.CreatedDate.ToString("yyyy- ↗
259     MM-dd HH:mm")</td>
260 <td>@(o.StartedDate?.ToString ↗
261     ("yyyy-MM-dd HH:mm") ?? "-")</td> ↗
262 <td>@(o.FinishedDate?.ToString ↗
263     ("yyyy-MM-dd HH:mm") ?? "-")</td> ↗
264 <td class="text-end">@ ↗
265     (o.CycleSeconds?.ToString()) ?? ↗
266     "-")</td> ↗
267 <td class="text- ↗
268     end">@o.QuantityProductFault</td> ↗
269 </tr>
270 }
271 }
272 </tbody>
273 </table>
274 </div>
275 <div class="card-body pt-3">
276 <div class="text-muted small">
277     Forsiden viser bare de viktigste nøkkeltallene. ↗
278     Detaljert sammenligning av Before og After finnes ↗
279     under Analyse.
280 </div>
281 </div>
282 </div>
283 </div>
284 <div class="col-12 col-xl-3">
285 <div class="portal-card">
286 <div class="card-body">
287 <div class="section-title mb-3">Hurtigvalg</div>
288 <div class="d-grid gap-2 quick-links">
289 <a class="btn btn-outline-primary" asp-page="/ ↗
290     CreateOrder">Ny ordre</a>

```

```
                Reports">Åpne rapport</a>
281            <a class="btn btn-outline-dark" asp-page="/
                Analyse">Åpne analyse</a>
282        </div>
283
284        <hr />
285
286        <div class="text-muted small">
287            Denne løsningen er laget som en enkel ERP for
                registrering, dokumentasjon og analyse av
                produksjonsordre.
288        </div>
289    </div>
290 </div>
291 </div>
292 </div>
```

```
1 using Microsoft.AspNetCore.Mvc.RazorPages;
2 using Microsoft.Data.SqlClient;
3
4 namespace ERP.Pages
5 {
6     public class IndexModel : PageModel
7     {
8         public string? ErrorMessage { get; set; }
9
10        private readonly string _connStr =
11            "Server=localhost\
12            \SQLEXPRESS;Database=ERP_DB;Trusted_Connection=True;TrustS
13            erverCertificate=True;";
14
15        public DashboardKpi Kpi { get; set; } = new();
16        public List<OrderRow> LatestOrders { get; set; } = new();
17
18        public void OnGet()
19        {
20            try
21            {
22                using var conn = new SqlConnection(_connStr);
23                conn.Open();
24
25                LoadKpi(conn);
26                LoadLatestOrders(conn);
27            }
28            catch (Exception ex)
29            {
30                ErrorMessage = "Feil ved lasting av forsiden: " +
31                    ex.Message;
32            }
33
34            private void LoadKpi(SqlConnection conn)
35            {
36                string sql = @"
37                SELECT
38                (SELECT COUNT(*)
39                FROM Orders
40                WHERE Status = 'Finished') AS CompletedOrders,
41                (SELECT COUNT(*)
42                FROM Orders
43                WHERE Status IN ('Pending', 'Active', 'Running')) AS
44                ActivePendingOrders,
45                (SELECT AVG(CAST(DATEDIFF(SECOND, StartedDate, FinishedDate) AS
46                FLOAT))
47                FROM Orders
48                WHERE Status = 'Finished'
49                AND StartedDate IS NOT NULL
50                AND FinishedDate IS NOT NULL) AS AvgCycleSeconds,
```



```

49
50     (SELECT AVG(CAST(ISNULL(QuantityProductFault, 0) AS FLOAT))
51     FROM Orders
52     WHERE Status = 'Finished') AS AvgQuantityProductFault;";
53
54     using var cmd = new SqlCommand(sql, conn);
55     using var r = cmd.ExecuteReader();
56
57     if (r.Read())
58     {
59         Kpi.CompletedOrders = r["CompletedOrders"] ==
            DBNull.Value ? 0 : Convert.ToInt32(r
            ["CompletedOrders"]);
60         Kpi.ActivePendingOrders = r["ActivePendingOrders"] ==
            DBNull.Value ? 0 : Convert.ToInt32(r
            ["ActivePendingOrders"]);
61         Kpi.AvgCycleSeconds = r["AvgCycleSeconds"] ==
            DBNull.Value ? null : Convert.ToDouble(r
            ["AvgCycleSeconds"]);
62         Kpi.AvgQuantityProductFault = r
            ["AvgQuantityProductFault"] == DBNull.Value ? null :
            Convert.ToDouble(r["AvgQuantityProductFault"]);
63     }
64 }
65
66 private void LoadLatestOrders(SqlConnection conn)
67 {
68     string sql = @"
69 SELECT TOP 10
70     OrderId,
71     Status,
72     CreatedDate,
73     StartedDate,
74     FinishedDate,
75     ISNULL(QuantityProductFault, 0) AS QuantityProductFault,
76     ISNULL(TestPhase, '') AS TestPhase,
77     CASE
78         WHEN StartedDate IS NOT NULL AND FinishedDate IS NOT NULL
79         THEN DATEDIFF(SECOND, StartedDate, FinishedDate)
80         ELSE NULL
81     END AS CycleSeconds
82 FROM Orders
83 ORDER BY OrderId DESC;";
84
85     using var cmd = new SqlCommand(sql, conn);
86     using var r = cmd.ExecuteReader();
87
88     while (r.Read())
89     {
90         LatestOrders.Add(new OrderRow
91         {
92             OrderId = Convert.ToInt32(r["OrderId"]),
93             Status = r["Status"]?.ToString() ?? "",

```

```
94         CreatedDate = Convert.ToDateTime(r["CreatedDate"]),
95         StartedDate = r["StartedDate"] == DBNull.Value ?
            null : Convert.ToDateTime(r["StartedDate"]),
96         FinishedDate = r["FinishedDate"] == DBNull.Value ?
            null : Convert.ToDateTime(r["FinishedDate"]),
97         QuantityProductFault = Convert.ToInt32(r
            ["QuantityProductFault"]),
98         TestPhase = r["TestPhase"]?.ToString() ?? "",
99         CycleSeconds = r["CycleSeconds"] == DBNull.Value ?
            null : Convert.ToInt32(r["CycleSeconds"])
100     });
101 }
102 }
103
104 public string FormatDouble(double? value, int decimals)
105 {
106     if (!value.HasValue)
107         return "-";
108
109     return value.Value.ToString("F" + decimals)
110         .Replace(",", "X")
111         .Replace(".", ",")
112         .Replace("X", ".");
113 }
114
115 public class DashboardKpi
116 {
117     public int CompletedOrders { get; set; }
118     public int ActivePendingOrders { get; set; }
119     public double? AvgCycleSeconds { get; set; }
120     public double? AvgQuantityProductFault { get; set; }
121 }
122
123 public class OrderRow
124 {
125     public int OrderId { get; set; }
126     public string Status { get; set; } = "";
127     public DateTime CreatedDate { get; set; }
128     public DateTime? StartedDate { get; set; }
129     public DateTime? FinishedDate { get; set; }
130     public int QuantityProductFault { get; set; }
131     public string TestPhase { get; set; } = "";
132     public int? CycleSeconds { get; set; }
133 }
134 }
135 }
```

```

1  @page
2  @model ERP.Pages.CreateOrderModel
3  @{
4      ViewData["Title"] = "Opprett ordre";
5  }
6
7  <div class="row justify-content-center">
8      <div class="col-12 col-lg-7">
9          <div class="card shadow-sm">
10             <div class="card-body">
11
12                 <h2 class="mb-4">Opprett ordre</h2>
13
14                 @if (!string.IsNullOrEmpty(Model.ErrorMessage))
15                 {
16                     <div class="alert alert-          ↗
17                         danger">@Model.ErrorMessage</div>
18                 }
19                 @if (!string.IsNullOrEmpty(Model.SuccessMessage))
20                 {
21                     <div class="alert alert-          ↗
22                         success">@Model.SuccessMessage</div>
23                 }
24
25                 <form method="post" class="d-grid gap-3">
26
27                     <div class="row">
28                         <div class="col">
29                             <label class="form-label">Rød (antall)</          ↗
30                             label>
31                             <input id="redQty" class="form-control"          ↗
32                                 type="number" name="RedQty"          ↗
33                                 min="0" max="6"          ↗
34                                 value="@Model.RedQty" />
35                         </div>
36
37                         <div class="col">
38                             <label class="form-label">Svart (antall)</          ↗
39                             label>
40                             <input id="blackQty" class="form-control"          ↗
41                                 type="number" name="BlackQty"          ↗
42                                 min="0" max="6"          ↗
43                                 value="@Model.BlackQty" />
44                         </div>
45                     </div>
46
47                     <div>
48                         <label class="form-label">Testfase</label>
49                         <select class="form-select" name="TestPhase">
50                             <option value="">Velg testfase</option>
51                             <option value="Before" selected="@          ↗
52                                 (Model.TestPhase == "Before")">Before</          ↗
53                             option>

```

```

44         <option value="After" selected="@
           (Model.TestPhase == "After")">After</
           option>
45     </select>
46     <small class="text-muted">
47         Before = før optimalisering, After = etter
           optimalisering
48     </small>
49 </div>
50
51 <div id="totalBox" class="alert alert-warning
           mb-0">
52     Total: <b id="totalValue">0</b>/12
53 </div>
54
55 <hr class="my-2" />
56
57 <h5 class="mb-2">Vann % per cup</h5>
58 <small class="text-muted">
59     Bare cup 1..Total vises. Resten skjules
           automatisk.
60 </small>
61
62 <div id="cupsGrid" class="row g-2">
63     @for (int i = 1; i <= 12; i++)
64     {
65         <div class="col-6 col-md-4 cup-item" data-
           cupno="@i">
66             <label class="form-label cup-label">
67                 Cup @i</label>
68
69                 <input class="form-control cup-water"
70                     type="number"
71                     name="CupWater[@(i-1)]"
72                     min="0" max="100"
73                     value="@((Model.CupWater[i-1]))" /
74             >
75         </div>
76     }
77 </div>
78
79 <button id="saveBtn" class="btn btn-primary mt-2"
           type="submit">
80     Lagre ordre
81 </button>
82
83 </form>
84
85 <small class="text-muted d-block mt-3">
86     Regel: Rød 0-6, Svart 0-6, Total 0-12.
87 </small>
</div>

```

```

88     </div>
89 </div>
90
91 @section Scripts {
92     <script>
93         const red = document.getElementById("redQty");
94         const black = document.getElementById("blackQty");
95         const totalValue = document.getElementById("totalValue");
96         const totalBox = document.getElementById("totalBox");
97         const saveBtn = document.getElementById("saveBtn");
98         const cupItems = document.querySelectorAll(".cup-item");
99
100         function clampNumberInput(input, min, max) {
101             let v = parseInt(input.value || "0", 10);
102             if (isNaN(v)) v = 0;
103             if (v < min) v = min;
104             if (v > max) v = max;
105             if (String(v) !== input.value) input.value = v;
106             return v;
107         }
108
109         function updateTotalAndCups() {
110             const r = clampNumberInput(red, 0, 6);
111             const b = clampNumberInput(black, 0, 6);
112
113             const total = r + b;
114             totalValue.textContent = total;
115
116             const okTotal = (total >= 0 && total <= 12);
117
118             if (okTotal) {
119                 totalBox.className = "alert alert-success mb-0";
120                 saveBtn.disabled = false;
121             } else {
122                 totalBox.className = "alert alert-danger mb-0";
123                 saveBtn.disabled = true;
124             }
125
126             let redIndex = 0;
127             let blackIndex = 0;
128
129             cupItems.forEach(item => {
130                 const cupNo = parseInt(item.dataset.cupno, 10);
131                 const label = item.querySelector(".cup-label");
132                 const input = item.querySelector("input");
133
134                 if (cupNo <= total) {
135                     item.style.display = "";
136
137                     if (cupNo <= r) {
138                         redIndex++;
139                         if (label) label.textContent = `Red ${redIndex} ➤`

```

```
140         } else {
141             blackIndex++;
142             if (label) label.textContent = `Black ${blackIndex}`;
143         }
144     } else {
145         item.style.display = "none";
146         if (input) input.value = 0;
147         if (label) label.textContent = `Cup ${cupNo}`;
148     }
149     });
150 }
151
152 red.addEventListener("input", updateTotalAndCups);
153 black.addEventListener("input", updateTotalAndCups);
154
155 updateTotalAndCups();
156 </script>
157 }
```

```
1 using Microsoft.AspNetCore.Mvc;
2 using Microsoft.AspNetCore.Mvc.RazorPages;
3 using Microsoft.Data.SqlClient;
4 using System.Globalization;
5
6 namespace ERP.Pages
7 {
8     public class AnalyseModel : PageModel
9     {
10         private const double CupWeightG = 14.61;
11         private const double WeightAt100PercentG = 28.62;
12         private const double NetWeightAt100PercentG =
13             WeightAt100PercentG - CupWeightG;
14
15         private readonly string _connStr =
16             "Server=localhost\
17             \SQLEXPRESS;Database=ERP_DB;Trusted_Connection=True;TrustS
18             erverCertificate=True;";
19
20         [BindProperty(SupportsGet = true)]
21         public string? SelectedPhase { get; set; }
22
23         [BindProperty(SupportsGet = true)]
24         public DateTime? FromDate { get; set; }
25
26         [BindProperty(SupportsGet = true)]
27         public DateTime? ToDate { get; set; }
28
29         [BindProperty(SupportsGet = true)]
30         public double Tolerance { get; set; } = 5.0;
31
32         public int TotalOrders { get; set; }
33         public int TotalCups { get; set; }
34         public int TotalProduced { get; set; }
35         public int TotalFaultProducts { get; set; }
36
37         public double AvgCycleTime { get; set; }
38         public double MinCycleTime { get; set; }
39         public double MaxCycleTime { get; set; }
40
41         public int QualityMeasurementCount { get; set; }
42         public int AcceptedCount { get; set; }
43         public double AvgWeightDifference { get; set; }
44         public double MaxAbsWeightDifference { get; set; }
45         public double MAE { get; set; }
46         public double MAPE { get; set; }
47         public double MSE { get; set; }
48         public double RMSE { get; set; }
49         public double AcceptanceRate { get; set; }
50
51         public List<CupAnalysisRow> CupRows { get; set; } = new();
52         public List<PhaseComparisonRow> PhaseComparisons { get; set; }
53             = new();
```

```
50
51     public List<string> CycleChartLabels { get; set; } = new();
52     public List<double> CycleChartValues { get; set; } = new();
53
54     public List<string> ErrorByCupChartLabels { get; set; } = new
55         ();
56     public List<double> ErrorByCupChartValues { get; set; } = new
57         ();
58
59     public string AvgCycleTimeFormatted => AvgCycleTime.ToString
60         ("N1", CultureInfo.InvariantCulture);
61     public string MinCycleTimeFormatted => MinCycleTime.ToString
62         ("N1", CultureInfo.InvariantCulture);
63     public string MaxCycleTimeFormatted => MaxCycleTime.ToString
64         ("N1", CultureInfo.InvariantCulture);
65
66     public string AvgWeightDifferenceFormatted =>
67         AvgWeightDifference.ToString("N2",
68             CultureInfo.InvariantCulture);
69     public string MaxAbsWeightDifferenceFormatted =>
70         MaxAbsWeightDifference.ToString("N2",
71             CultureInfo.InvariantCulture);
72     public string MAEFormatted => MAE.ToString("N2",
73             CultureInfo.InvariantCulture);
74     public string MAPEFormatted => MAPE.ToString("N2",
75             CultureInfo.InvariantCulture) + "%";
76     public string MSEFormatted => MSE.ToString("N2",
77             CultureInfo.InvariantCulture);
78     public string RMSEFormatted => RMSE.ToString("N2",
79             CultureInfo.InvariantCulture);
80     public string AcceptanceRateFormatted =>
81         AcceptanceRate.ToString("N1", CultureInfo.
82             InvariantCulture) + "%";
83
84     public string CycleImprovementText
85     {
86         get
87         {
88             var before = PhaseComparisons.FirstOrDefault(x =>
89                 x.TestPhase == "Before");
90             var after = PhaseComparisons.FirstOrDefault(x =>
91                 x.TestPhase == "After");
92
93             if (before == null || after == null ||
94                 before.AvgCycleTime <= 0)
95                 return "-";
96
97             var improvement = ((before.AvgCycleTime -
98                 after.AvgCycleTime) / before.AvgCycleTime) * 100.0;
99             return improvement.ToString("N1",
100                 CultureInfo.InvariantCulture) + "%";
101         }
102     }
103 }
```



```
83
84     public string MapeImprovementText
85     {
86         get
87         {
88             var before = PhaseComparisons.FirstOrDefault(x =>
89                 x.TestPhase == "Before");
90             var after = PhaseComparisons.FirstOrDefault(x =>
91                 x.TestPhase == "After");
92
93             if (before == null || after == null)
94                 return "-";
95
96             var improvement = before.MAPE - after.MAPE;
97             return improvement.ToString("N2",
98                 CultureInfo.InvariantCulture) + " prosentpoeng";
99         }
100     }
101
102     public void OnGet()
103     {
104         if (Tolerance < 0)
105             Tolerance = 5.0;
106
107         LoadAnalysis();
108     }
109
110     private void LoadAnalysis()
111     {
112         using var connection = new SqlConnection(_connStr);
113         connection.Open();
114
115         LoadCupRows(connection);
116         CalculateSummary();
117         CalculatePhaseComparison();
118         BuildCharts();
119     }
120
121     private void LoadCupRows(SqlConnection connection)
122     {
123         var sql = @"
124 SELECT
125     o.OrderId,
126     o.CreatedDate,
127     o.TestPhase,
128     o.QuantityProduced,
129     o.QuantityProductFault,
130     oc.CupNo,
131     oc.Color,
132     oc.WaterPercent,
133     oc.WeightDifference,
134     oc.RealWeight,
135     oc.TimeSpentMinutes
```

```
133 FROM OrderCups oc
134 INNER JOIN Orders o ON o.OrderId = oc.OrderId
135 WHERE o.Status = 'Finished'
136 ";
137
138     var filters = new List<string>();
139
140     if (!string.IsNullOrEmpty(SelectedPhase))
141         filters.Add("o.TestPhase = @TestPhase");
142
143     if (FromDate.HasValue)
144         filters.Add("CAST(o.CreatedDate AS date) >= @FromDate");
145
146     if (ToDate.HasValue)
147         filters.Add("CAST(o.CreatedDate AS date) <= @ToDate");
148
149     if (filters.Count > 0)
150         sql += " AND " + string.Join(" AND ", filters);
151
152     sql += " ORDER BY o.OrderId, oc.CupNo";
153
154     using var cmd = new SqlCommand(sql, connection);
155
156     if (!string.IsNullOrEmpty(SelectedPhase))
157         cmd.Parameters.AddWithValue("@TestPhase",
158                                     SelectedPhase!);
159
160     if (FromDate.HasValue)
161         cmd.Parameters.AddWithValue("@FromDate",
162                                     FromDate.Value.Date);
163
164     if (ToDate.HasValue)
165         cmd.Parameters.AddWithValue("@ToDate",
166                                     ToDate.Value.Date);
167
168     using var reader = cmd.ExecuteReader();
169     while (reader.Read())
170     {
171         var waterPercent = reader["WaterPercent"] ==
172             DBNull.Value ? 0 : Convert.ToDouble(reader
173             ["WaterPercent"]);
174
175         var weightDifference = reader["WeightDifference"] ==
176             DBNull.Value ? (double?)null : Convert.ToDouble
177             (reader["WeightDifference"]);
178
179         var realWeight = reader["RealWeight"] == DBNull.
180             Value ? (double?)null : Convert.ToDouble(reader
181             ["RealWeight"]);
182
183         var timeSpentMinutes = reader["TimeSpentMinutes"] ==
184             DBNull.Value ? (decimal?)null : Convert.ToDecimal
185             (reader["TimeSpentMinutes"]);
186
187         var targetNetWeight = (waterPercent / 100.0) *
```

```

        NetWeightAt100PercentG;
174         double? errorPercent = null;
175
176         if (weightDifference.HasValue && realWeight.
            HasValue && realWeight.Value > CupWeightG)
177             errorPercent = Math.Abs(weightDifference.Value) /
                (realWeight.Value - CupWeightG) * 100.0;
178         else if (weightDifference.HasValue &&
            targetNetWeight > 0)
179             errorPercent = Math.Abs(weightDifference.Value) /
                targetNetWeight * 100.0;
180
181         CupRows.Add(new CupAnalysisRow
182         {
183             OrderId = Convert.ToInt32(reader["OrderId"]),
184             TestPhase = reader["TestPhase"]?.ToString() ?? "",
185             CreatedDate = reader["CreatedDate"] ==
                DBNull.Value ? null : Convert.ToDateTime(reader
                ["CreatedDate"]),
186             QuantityProduced = reader["QuantityProduced"] ==
                DBNull.Value ? 0 : Convert.ToInt32(reader
                ["QuantityProduced"]),
187             QuantityProductFault = reader
                ["QuantityProductFault"] == DBNull.Value ? 0 :
                Convert.ToInt32(reader["QuantityProductFault"]),
188             CupNo = reader["CupNo"] == DBNull.Value ? 0 :
                Convert.ToInt32(reader["CupNo"]),
189             Color = reader["Color"]?.ToString() ?? "",
190             WaterPercent = waterPercent,
191             WeightDifference = weightDifference,
192             RealWeight = realWeight,
193             ErrorPercent = errorPercent,
194             CycleTimeSeconds = CalculateCycleTimeSeconds
                (timeSpentMinutes)
195         });
196     }
197 }
198
199 private void CalculateSummary()
200 {
201     TotalOrders = CupRows.Select(x => x.OrderId).Distinct
        ().Count();
202     TotalCups = CupRows.Count;
203
204     TotalProduced = CupRows
        .GroupBy(x => x.OrderId)
205         .Select(g => g.First().QuantityProduced)
206         .Sum();
207
208     TotalFaultProducts = CupRows
        .GroupBy(x => x.OrderId)
209         .Select(g => g.First().QuantityProductFault)
210         .Sum();
211
212

```

```

213
214         var cycleRows = CupRows.Where(x => x.CycleTimeSeconds > 0).ToList();
215         AvgCycleTime = cycleRows.Count > 0 ? cycleRows.Average(x => x.CycleTimeSeconds) : 0;
216         MinCycleTime = cycleRows.Count > 0 ? cycleRows.Min(x => x.CycleTimeSeconds) : 0;
217         MaxCycleTime = cycleRows.Count > 0 ? cycleRows.Max(x => x.CycleTimeSeconds) : 0;
218
219         var qualityRows = CupRows.Where(x => x.WeightDifference.HasValue).ToList();
220         QualityMeasurementCount = qualityRows.Count;
221         AcceptedCount = qualityRows.Count(x => Math.Abs(x.WeightDifference!.Value) <= Tolerance);
222         AvgWeightDifference = qualityRows.Count > 0 ? qualityRows.Average(x => x.WeightDifference!.Value) : 0;
223         MAE = qualityRows.Count > 0 ? qualityRows.Average(x => Math.Abs(x.WeightDifference!.Value)) : 0;
224         MSE = qualityRows.Count > 0 ? qualityRows.Average(x => Math.Pow(x.WeightDifference!.Value, 2)) : 0;
225         RMSE = Math.Sqrt(MSE);
226         MaxAbsWeightDifference = qualityRows.Count > 0 ? qualityRows.Max(x => Math.Abs(x.WeightDifference!.Value)) : 0;
227         AcceptanceRate = QualityMeasurementCount > 0 ? AcceptedCount * 100.0 / QualityMeasurementCount : 0;
228
229         var mapeRows = CupRows.Where(x => x.ErrorPercent.HasValue).ToList();
230         MAPE = mapeRows.Count > 0 ? mapeRows.Average(x => x.ErrorPercent!.Value) : 0;
231     }
232
233     private void CalculatePhaseComparison()
234     {
235         PhaseComparisons = CupRows
236             .Where(x => !string.IsNullOrWhiteSpace(x.TestPhase))
237             .GroupBy(x => x.TestPhase)
238             .Select(g =>
239             {
240                 var qualityRows = g.Where(x => x.WeightDifference.HasValue).ToList();
241                 var cycleRows = g.Where(x => x.CycleTimeSeconds > 0).ToList();
242                 var mapeRows = g.Where(x => x.ErrorPercent.HasValue).ToList();
243
244                 return new PhaseComparisonRow
245                 {
246                     TestPhase = g.Key,
247                     CupCount = g.Count(),
248                     OrderCount = g.Select(x => x.OrderId).Distinct

```

```

        ().Count(),
249         AvgCycleTime = cycleRows.Count > 0 ?
            cycleRows.Average(x => x.CycleTimeSeconds) :
            0,
250         MAE = qualityRows.Count > 0 ?
            qualityRows.Average(x => Math.Abs
            (x.WeightDifference!.Value)) : 0,
251         MAPE = mapeRows.Count > 0 ? mapeRows.Average(x
            => x.ErrorPercent!.Value) : 0,
252         AcceptanceRate = qualityRows.Count > 0
253             ? qualityRows.Count(x => Math.Abs
                (x.WeightDifference!.Value) <= Tolerance)
                * 100.0 / qualityRows.Count
                : 0
254     };
255     });
256     })
257     .OrderBy(x => x.TestPhase)
258     .ToList();
259 }
260
261 private void BuildCharts()
262 {
263     var cycleRows = CupRows.Where(x => x.CycleTimeSeconds >
        0).OrderBy(x => x.OrderId).ThenBy(x => x.CupNo).ToList();
264
265     CycleChartLabels = cycleRows
266         .Select(x => $"O{x.OrderId}-K{x.CupNo}")
267         .ToList();
268
269     CycleChartValues = cycleRows
270         .Select(x => Math.Round(x.CycleTimeSeconds, 2))
271         .ToList();
272
273     var errorByCup = CupRows
274         .Where(x => x.ErrorPercent.HasValue)
275         .GroupBy(x => x.CupNo)
276         .OrderBy(g => g.Key)
277         .Select(g => new { CupNo = g.Key, Mape = g.Average(
            x => x.ErrorPercent!.Value) })
278         .ToList();
279
280     ErrorByCupChartLabels = errorByCup
281         .Select(x => $"Kopp {x.CupNo}")
282         .ToList();
283
284     ErrorByCupChartValues = errorByCup
285         .Select(x => Math.Round(x.Mape, 2))
286         .ToList();
287 }
288
289 private static double CalculateCycleTimeSeconds(decimal?
    timeSpentMinutes)
290 {

```

```
291         if (timeSpentMinutes.HasValue && timeSpentMinutes.Value > 0)
292             return (double)timeSpentMinutes.Value * 60.0;
293
294         return 0;
295     }
296
297     public class CupAnalysisRow
298     {
299         public int OrderId { get; set; }
300         public string TestPhase { get; set; } = "";
301         public DateTime? CreatedDate { get; set; }
302         public int QuantityProduced { get; set; }
303         public int QuantityProductFault { get; set; }
304         public int CupNo { get; set; }
305         public string Color { get; set; } = "";
306         public double WaterPercent { get; set; }
307         public double? WeightDifference { get; set; }
308         public double? RealWeight { get; set; }
309         public double? ErrorPercent { get; set; }
310         public double CycleTimeSeconds { get; set; }
311
312         public string CreatedDateText => CreatedDate?.ToString(
313             ("dd.MM.yyyy HH:mm:ss") ?? "-");
314         public string WeightDifferenceText =>
315             WeightDifference.HasValue ?
316             WeightDifference.Value.ToString("N2",
317                 CultureInfo.InvariantCulture) : "-";
318         public string RealWeightText => RealWeight.HasValue ?
319             RealWeight.Value.ToString("N2",
320                 CultureInfo.InvariantCulture) : "-";
321         public string ErrorPercentText => ErrorPercent.HasValue ?
322             ErrorPercent.Value.ToString("N2",
323                 CultureInfo.InvariantCulture) + "%" : "-";
324     }
325
326     public class PhaseComparisonRow
327     {
328         public string TestPhase { get; set; } = "";
329         public int OrderCount { get; set; }
330         public int CupCount { get; set; }
331         public double AvgCycleTime { get; set; }
332         public double MAE { get; set; }
333         public double MAPE { get; set; }
334         public double AcceptanceRate { get; set; }
335     }
336 }
```